

Synchronization

Warm-Up Example

```
int main() {
    int t;
    for (t = 1; t ≤ 10; t++) {
        num = 0;
        pthread_t thread_inc, thread_dec;
        pthread_create(&thread_inc, NULL, &inc, NULL);
        pthread_create(&thread_dec, NULL, &dec, NULL);
        pthread_join(thread_inc, NULL);
        pthread_join(thread_dec, NULL);

        printf("#%d, %d\n", t, num);
    }
    return 0;
}
```

```
#include <pthread.h>
#include <stdio.h>

int num;

void* inc(void* arg) {
    int i;
    for(i=0; i<10000; i++)
        num++;
    pthread_exit(NULL);
}

void *dec(void *arg) {
    int i;
    for(i=0; i<10000; i++)
        num--;
    pthread_exit(NULL);
}
```

Warm-Up Example

- Why are the results random?

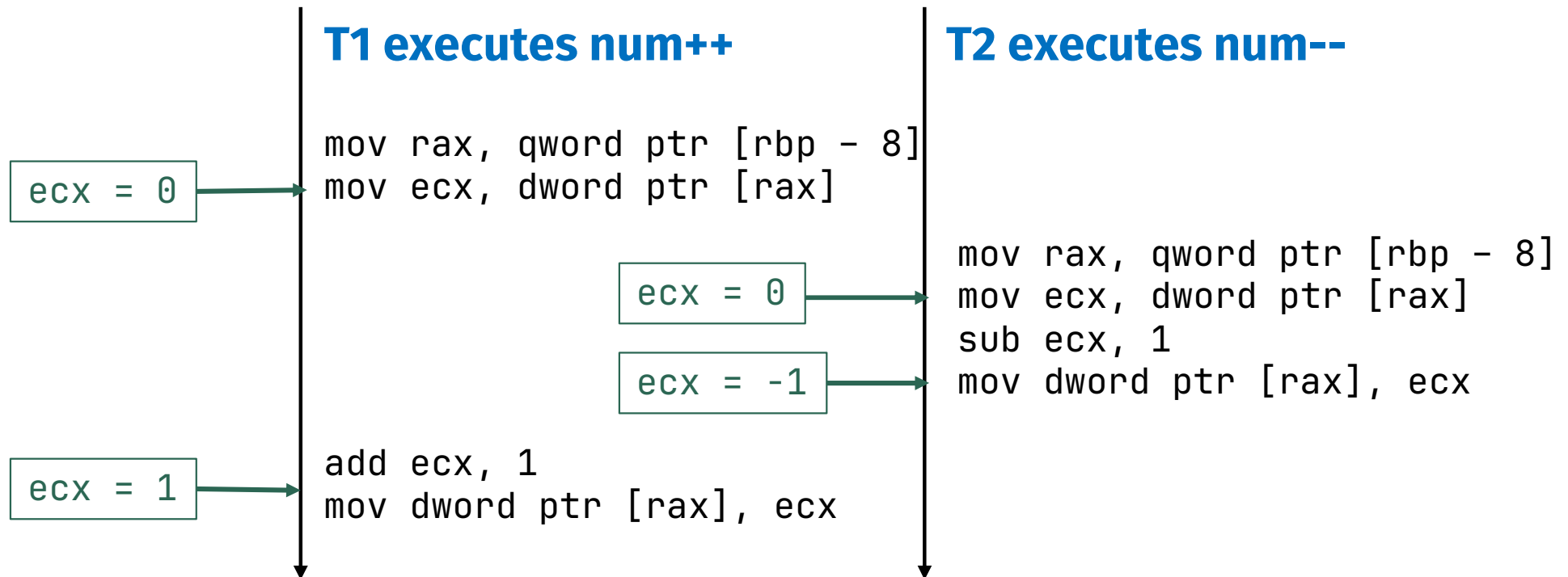
```
 ./a.out  
#1, 0  
#2, -5628  
#3, -5774  
#4, 0  
#5, -3963  
#6, -2451  
#7, -6160  
#8, -2185  
#9, 0  
#10, -1041
```

Mutex (1)

- **Mutex is an abbreviation for “mutual exclusion”**
 - Primary means of implementing **thread synchronization**
 - » Protects shared data when **multiple writes** occurs
 - A mutex variable acts like a “lock” protecting access to **shared data**
 - » Only one thread can lock (or own) a mutex variable at any given time
 - » Even if several threads try to lock a mutex, **only one thread will be successful** and **other threads are blocked** until the owner releases the lock
 - Mutex is used to prevent “**race conditions**”
 - » race condition: anomalous behavior due to unexpected critical dependence on the relative timing of events

Mutex (2)

num = 0



Creating/Destroying Mutexes

▪ Static initialization

- `pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;`

▪ Dynamic initialization

- `pthread_mutex_t m;`
- `pthread_mutex_init(&m, (pthread_mutexattr_t *) NULL);`

▪ Destroying a mutex

- `pthread_mutex_destroy(&m);`
- Destroys a mutex object, freeing the resources it might hold

Using Mutexes

- `int pthread_mutex_lock(pthread_mutex_t* mutex)`
 - **Acquire a lock** on the specified **mutex** variable
 - If the **mutex** is already locked by another thread, block the calling thread until the **mutex** is unlocked
- `int pthread_mutex_unlock(pthread_mutex_t* mutex)`
 - Unlock a **mutex** if called by the owning thread
- `int pthread_mutex_trylock(pthread_mutex_t* mutex)`
 - Attempt to lock a **mutex**
 - If the **mutex** is already locked, return immediately with a “**busy**” error code

Example

```
#include <pthread.h>
#include <stdio.h>

int num;
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;

void* inc(void* arg) {
    int i;
    for (i = 0; i < 10000; i++) {
        pthread_mutex_lock(&m);
        num++;
        pthread_mutex_unlock(&m);
    }
    pthread_exit(NULL);
}

void* dec(void* arg) {
    int i;
    for (i = 0; i < 10000; i++) {
        pthread_mutex_lock(&m);
        num--;
        pthread_mutex_unlock(&m);
    }
    pthread_exit(NULL);
}
```



./a.out

```
#1, 0
#2, 0
#3, 0
#4, 0
#5, 0
#6, 0
#7, 0
#8, 0
#9, 0
#10, 0
```

Condition Variables (1)

■ Another way for thread synchronization

- While mutexes implement synchronization by controlling thread access to data, condition variables allow threads to synchronize based upon the actual value of data
- Without condition variables, the programmer would need to have threads continually polling to check if the condition is met
 - » This can be very resource consuming since the thread would be continuously busy in this activity
- A condition variable is always used in conjunction with a mutex lock

Condition Variables (2)

■ How condition variables work

- A thread locks a mutex associated with a condition variable
- The thread tests the condition to see if it can proceed
- If it can
 - » Your thread does its work
 - » Your thread unlocks the mutex
- If it cannot
 - » The thread sleeps; the mutex is automatically released
 - » Some other threads signals the condition variable
 - » Your thread wakes up from waiting with the mutex automatically locked,
and it does its work
 - » Your thread releases the mutex when it's done

Creating / Destroying CV

▪ Static initialization

- `pthread_cond_t cond = PTHREAD_COND_INITIALIZER;`

▪ Dynamic initialization

- `pthread_cond_t cond;`
`pthread_cond_init(&cond,`
`(pthread_condattr_t*)NULL);`

▪ Destroying a condition variable

- `pthread_cond_destroy(&cond);`
- Destroys a condition variable, freeing the resources it might hold

Using Condition Variables

- `int pthread_cond_wait(pthread_cond_t* cond, pthread_mutex_t* mutex)`
 - Blocks the calling thread until the specified condition is signaled
 - This should be called while mutex is locked, and it will automatically release the mutex while it waits
- `int pthread_cond_signal(pthread_cond_t* cond)`
 - Signals another thread which is waiting on the condition variable
 - Calling thread should have a lock
- `int pthread_cond_broadcast(pthread_cond_t* cond)`
 - Used if more than one thread is in a blocking wait state

C.V. Example

```
action() {  
    pthread_mutex_lock(&mutex);  
    while (x  $\neq$  0)  
        pthread_cond_wait(cond, mutex);  
    real_action();  
    pthread_mutex_unlock(&mutex);  
}
```

```
counter() {  
    pthread_mutex_lock(&mutex);  
    x--;  
    if (x  $=$  0)  
        pthread_cond_signal(cond);  
    pthread_mutex_unlock(&mutex);  
}
```

Thread Safety (1)

▪ Thread-safe

- Functions called from a thread must be **thread-safe**
- We identify four (non-disjoint) classes of thread-unsafe functions:
 - » Class 1: Failing to protect shared variables
 - » Class 2: Relying on persistent state across invocations
 - » Class 3: Returning a pointer to a static variable
 - » Class 4: Calling thread-unsafe functions

Thread Safety (2)

▪ Class 1: Failing to protect shared variables

- Fix: Use mutex operations
- Issue: Synchronization operations will slow down code

```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
int cnt = 0;

/* Thread routine */
void* count(void* arg) {
    int i;

    for (i = 0; i < N_ITERATIONS; i++) {
        pthread_mutex_lock(&lock);
        cnt++;
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}
```

Thread Safety (3)

- **Class 2: Relying on persistent state across multiple function invocations**
 - Random number generator relies on static state
 - Fix: Rewrite function so that caller passes all necessary states

```
/* rand - return pseudo-random integer on 0..32767 */  
static unsigned int next = 1;  
  
int rand(void) {  
    next = next * 1103515245 + 12345;  
    return (unsigned int)(next / 65536) % 32768;  
}  
  
/* srand - set seed for rand() */  
void srand(unsigned int seed) {  
    next = seed;  
}
```

Thread Safety (4)

- **Class 3: Returning a ptr to a *static variable***

- **Fixes:**

1. Rewrite code so caller passes pointer to **struct**

- » Issue: Requires changes in caller and callee

```
struct hostent* gethostbyname(  
    char* name) {  
    static struct hostent h;  
    // contact DNS and fill in h  
    return &h;  
}
```

```
hostp = malloc(...);  
gethostbyname_r(name, hostp, ...);
```

2. Lock-and-copy

- » Issue: Requires only simple changes in caller (and none in callee)

- » However, caller must **free memory**

```
struct hostent* gethostbyname_ts(  
    char* name) {  
    struct hostent* unshared = malloc(...);  
    pthread_mutex_lock(&lock); /* lock */  
    shared = gethostbyname(name);  
    *unshared = *shared; /* copy */  
    pthread_mutex_unlock(&lock);  
    return q;  
}
```

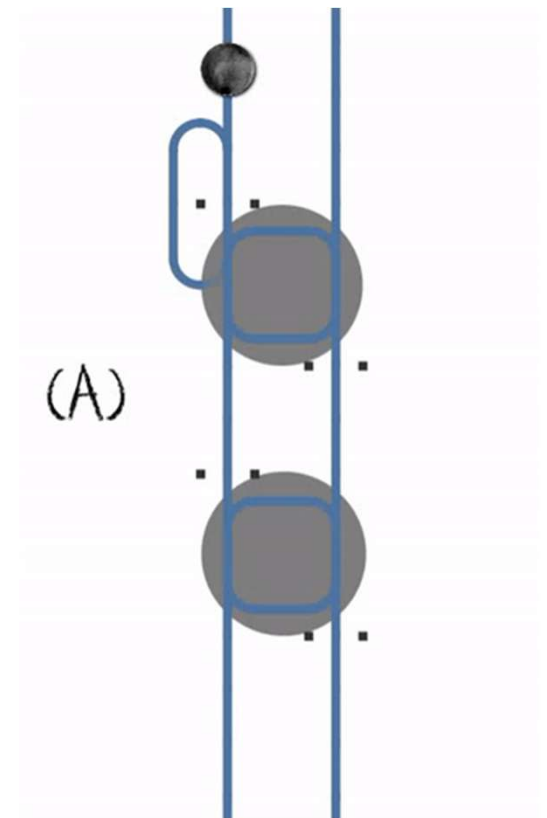
Thread Safety (5)

- **Class 4: Calling thread-unsafe functions**
 - Calling one thread-unsafe function makes an entire function thread-unsafe
 - Fix: Modify the function so it calls only thread-safe functions

Deadlock

▪ Deadlock

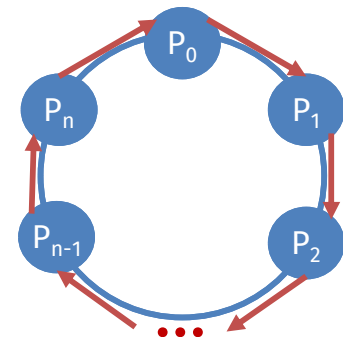
- In concurrent computing, a **deadlock** is a state in which each member of a group is **waiting for another member**, including itself, to take action such as sending a message or more commonly, **releasing a lock**



Deadlock

▪ Necessary condition:

- The following four conditions must hold for a deadlock to possibly occur, but this does not guarantee a deadlock.
- Mutual exclusion
 - Only one process at a time can use a resource
- Hold and wait
 - A process holding at least one resource is waiting to acquire additional resources held by other processes
- No preemption
 - A resource can be released only voluntarily by the process holding it, after that process has completed its task
- Circular wait
 - Set $\{P_0, P_1, \dots, P_n\}$ of waiting processes such that P_0 is waiting for a resource that is held by P_1 , P_1 is waiting for a resource that is held by P_2 , ... P_{n-1} is waiting for a resource that is held by P_n , and P_n is waiting for a resource that is held by P_0



Deadlock

▪ Solution

- Ensure that the system will never enter a deadlock state
- Allow the system to enter a deadlock state and then recover
- Ignore the problem and pretend that deadlocks never occur in the system; used by most operating systems, including UNIX

Deadlock Prevention

▪ Mutual exclusion

- Not required for sharable resources
- Must hold for non-sharable resources

▪ Hold and wait

- Must guarantee that whenever a process requests a resource, it does not hold any other resources
- Require process to request and be allocated all its resources at once before it begins execution, or allow process to request resources only when the process has none
- Low resource utilization; starvation possible

Deadlock Prevention

■ No preemption

- If a process that is holding some resources requests another resource that cannot be immediately allocated to it, then all resources currently being held are released.
- Preempted resources are added to the list of resources for other processes who are waiting for the resources
- Process will be restarted only when it can regain its old resources, as well as the new ones that are requested by the process.

■ Circular wait

- Impose a total ordering of all resource types and require that each process requests resources in an increasing order

Deadlock Example

```
#include <fcntl.h>
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <unistd.h>

int main() {
    // make thread and start thread
    pthread_t thread_get, thread_set;
    pthread_create(&thread_set, NULL, &setter, NULL);
    nanosleep(&(struct timespec){0, 500000000}, NULL);
    pthread_create(&thread_get, NULL, &getter, NULL);
    pthread_join(thread_set, NULL);
    pthread_join(thread_get, NULL);
    return 0;
}
```

Deadlock Example

```
pthread_mutex_t file_mutex[] = { // 5 mutex locks for 5 files
    PTHREAD_MUTEX_INITIALIZER, PTHREAD_MUTEX_INITIALIZER,
    PTHREAD_MUTEX_INITIALIZER, PTHREAD_MUTEX_INITIALIZER,
    PTHREAD_MUTEX_INITIALIZER}; // note: static initialization is okay
because file_mutex is a global variable...not okay if it is in heap

int get_data(char* path) { // path as an argument
    int fd = open(path, O_RDONLY);
    char buf[10];
    memset(buf, 0, 10);
    read(fd, buf, 10); // read 10 bytes from fd (path)
    return (int)strtol(buf, NULL, 10);
}

void set_data(char* path, int data) { // path, data as arguments
    int fd = open(path, O_WRONLY | O_CREAT, 0777);
    char buf[10];
    memset(buf, 0, 10);
    sprintf(buf, "%d", data); // set buf with data
    for (int i = 0; i < 10; i++) { // write 1 byte 10 times
        if (buf[i] != 0) write(fd, buf + i, 1);
        else break;
    }
}
```

Deadlock Example

```
void* setter(void* tid) {
    char path[30];
    int count = 0;
    while (1) {
        // lock all 5 files for consistency
        for (int i = 4; i ≥ 0; i--) { // lock in descending order
            pthread_mutex_lock(&(file_mutex[i]));
        }
        for (int i = 4; i ≥ 0; i--) { // start from 4.txt to 0.txt
            memset(path, 0, 30);
            sprintf(path, "%d.txt", i);
            set_data(path, count); // set all 5 files with count
        }
        for (int i = 4; i ≥ 0; i--) { // unlock in descending order
            pthread_mutex_unlock(&(file_mutex[i]));
        }
        count = (count + 1) % 10; // set count to 0, 1, ... 9, repeatedly
        nanosleep(&(struct timespec){0, 100000000}, NULL);
    }
}
```

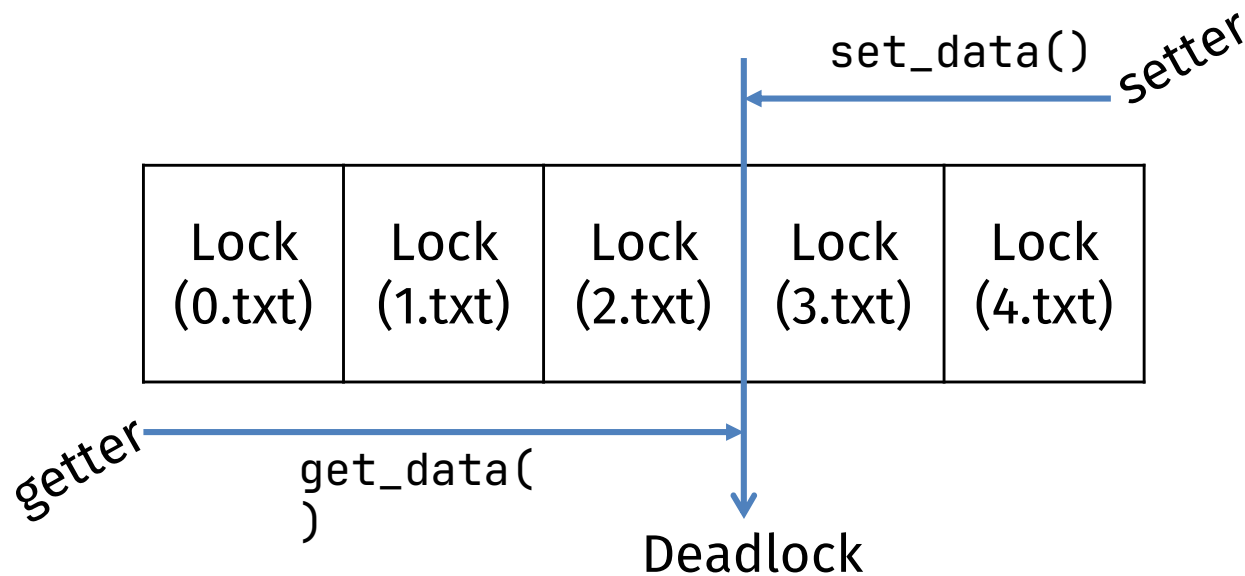
Deadlock Example

```
void* getter(void* tid) {
    char path[30];
    while (1) {
        int count = 0;
        // lock all 5 files for consistency
        for (int i = 0; i < 5; i++) { // lock in ascending order
            pthread_mutex_lock(&(file_mutex[i]));
        }
        for (int i = 0; i < 5; i++) { // start from 0.txt to 4.txt
            memset(path, 0, 30);
            sprintf(path, "%d.txt", i);
            count += get_data(path); // accumulate return values to count
        }
        for (int i = 0; i < 5; i++) { // unlock in ascending order
            pthread_mutex_unlock(&(file_mutex[i]));
        }
        printf("get_data: %d\n", count); // print sum of count values
        nanosleep(&(struct timespec){0, 100000000}, NULL);
    }
}
```

Deadlock Example

■ Execution

- Stop getting data
-> Deadlock



```
./a.out
get_data: 25
get_data: 30
get_data: 35
get_data: 40
get_data: 45
get_data: 0
get_data: 5
get_data: 10
get_data: 15
get_data: 20
get_data: 25
get_data: 30
get_data: 35
get_data: 40
get_data: 45
get_data: 0
get_data: 5
get_data: 10
get_data: 15
get_data: 20
get_data: 25
get_data: 30
get_data: 35
get_data: 40
get_data: 45
get_data: 0
get_data: 5
```

Lab Exercise

- **Make producer and consumer working at the same time**
 - The producer puts the number of LOOP in the queue
 - The consumer gets the number of LOOP in the queue
 - If the queue is full, producer can't put it and if queue is empty, consumer can't get it
 - You should use a synchronization mechanism using CV

Producer-Consumer (1)

```
#include <pthread.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    size_t put_index;    // rear
    size_t get_index;    // front
    size_t length;       // size
    size_t capacity;
    size_t total_n_iterations;
    pthread_mutex_t lock;
    pthread_cond_t not_full;
    pthread_cond_t not_empty;
    size_t data[];
} queue_t;

void* produce(void* args);
void* consume(void* args);
void put_data(queue_t* queue, size_t data);
size_t get_data(queue_t* queue);
```

Producer-Consumer (2)

```
queue_t* qinit(size_t total_n_iterations, size_t capacity) {
    queue_t* queue = (queue_t*)malloc(sizeof(queue_t) +
                                      sizeof(size_t[capacity]));
    queue->capacity = capacity;
    queue->total_n_iterations = total_n_iterations;
    queue->put_index = queue->get_index = queue->length = 0;
    pthread_mutex_init(&queue->lock, NULL);
    pthread_cond_init(&queue->not_full, NULL);
    pthread_cond_init(&queue->not_empty, NULL);
    return queue;
}

void qdelete(queue_t* queue) {
    pthread_mutex_destroy(&queue->lock);
    pthread_cond_destroy(&queue->not_full);
    pthread_cond_destroy(&queue->not_empty);
    free(queue);
}
```

Producer-Consumer (4)

```
int main(int argc, char* argv[]) {
    if (argc != 3) {
        fprintf(stderr,
            "usage: %s <total_n_iterations> <queue_size>\n",
            argv[0]);
        return 1;
    }
    queue_t* queue =
        qinit(strtoul(argv[1], NULL, 10), strtoul(argv[2], NULL, 10));

    pthread_t producer, consumer;
    pthread_create(&producer, NULL, produce, (void*)queue);
    pthread_create(&consumer, NULL, consume, (void*)queue);

    pthread_join(producer, NULL);
    pthread_join(consumer, NULL);

    qdelete(queue);
}
```

Lab Exercise

- **Make put_data and get_data function.**
 - `void put_data(queue_t* queue, size_t data);`
 - `size_t get_data(queue_t* queue);`
- **All produced data must be consumed**
 - You should use a synchronization mechanism using CV
 - Amount of data to be produced is total_n_iterations (argv[1]) while the capacity of the queue is capacity (argv[2]).
 - The number of consecutive lines with “get/put” does not have to be equal to the capacity.
 - i.e., (put 0, put 1, put 2, get 0, put 3, get 1, get 2, put 4, put 5, get 3, put 6, etc.) if capacity is 5.

```
put data 0 to queue
put data 1 to queue
put data 2 to queue
put data 3 to queue
put data 4 to queue
get data 0 from queue
get data 1 from queue
put data 5 to queue
put data 6 to queue
get data 2 from queue
get data 3 from queue
get data 4 from queue
get data 5 from queue
get data 6 from queue
put data 7 to queue
put data 8 to queue
put data 9 to queue
put data 10 to queue
put data 11 to queue
get data 7 from queue
get data 8 from queue
get data 9 from queue
get data 10 from queue
get data 11 from queue
put data 12 to queue
put data 13 to queue
put data 14 to queue
put data 15 to queue
put data 16 to queue
get data 12 from queue
get data 13 from queue
get data 14 from queue
get data 15 from queue
get data 16 from queue
```

Exercise Submission

- **Submit your source code and Makefile in iCampus**
 - Bundle **source code** and **Makefile** with tar command
`$ tar -caf STUDENTID_w13.tar.zst week13`
 - We'll grade your submission with **make**
 - » Should generate a w13 executable
 - » If compilation fails, your points for this exercise will be zero